Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Теоретическая информатика и компьютерные технологии»

Лабораторная работа № 13

по курсу «Методы оптимизации»

«Метод штрафных функций»

Студент группы ИУ9-82Б Шемякин В.А.

Преподаватель Посевин Д. П.

Москва 2026

1 Задание

Реализовавть метод штрафных функций

2 Результаты

Исходный код программы представлен в листинге 1.

```
using LinearAlgebra
using Plots
function golden_section_search(  , a, b; tol=1e-8, max_iter=10_000)
    a, b = (a <= b) ? (a, b) : (b, a)
    gr = (sqrt(5) - 1) / 2
    c = b - gr * (b - a)
    d = a + gr * (b - a)
    fc =   (c)
    fd =   (d)
    it = 0
    while (b - a) > tol && it < max_iter
        if fc <= fd
            b = d
            d = c
            fd = fc
            c = b - gr * (b - a)
            fc =   (c)
        else
            a = c
            c = d
            fc = fd
            d = a + gr * (b - a)
            fd =   (d)
        end
        it += 1
    end
    x = (a + b) / 2
    return x,   (x), it
end

H(h) = h * h
G(g) = (g > 0) ? (g * g) : 0.0

function Q(x, r, f, h_list::Vector{Function}, g_list::Vector{
    Function}, H, G)
```

```
    m = length(h_list)
    k = m + length(g_list)
    k == length(r) || throw(ArgumentError("length(r) must be m + (k-m)"))
    ans = 0
    for j in 1:m
        ans += r[j] * H(h_list[j](x))
    end
    for j in 1:(k - m)
        ans += r[m + j] * G(g_list[j](x))
    end
    ans += f(x)
    return ans
end

function violation(x, h_list::Vector{Function}, g_list::Vector{Function}, H_fun, G_fun)
    s = 0.0
    for h in h_list
        s += H_fun(h(x))
    end
    for g in g_list
        s += G_fun(g(x))
    end
    return s
end

function exterior_penalty_method_1d(f, h_list::Vector{Function}, g_list::Vector{Function}, x0, a, b;
    r0=1.0, beta=10.0, tol_x=1e-10, tol_violation=1e-12, max_outer=50, inner_tol=1e-12, H_fun=H, G_fun=G)
    m = length(h_list)
    k = m + length(g_list)
    r = fill(r0, k)
    x_prev = clamp(x0, min(a, b), max(a, b))
    outer = 0
    while outer < max_outer
            = x -> Q(x, r, f, h_list, g_list, H_fun, G_fun)
        x_new, _, _ = golden_section_search(  , a, b; tol=inner_tol)
        x_new = clamp(x_new, min(a, b), max(a, b))
        v = violation(x_new, h_list, g_list, H_fun, G_fun)
        if abs(x_new - x_prev) <= tol_x && v <= tol_violation
            x_prev = x_new
            break
```

```
        end
        x_prev = x_new
        r .*= beta
        outer += 1
    end
    x = x_prev
    return x, f(x), violation(x, h_list, g_list, H_fun, G_fun), r, outer
end

function backtracking_step(f, x, fx, g;  0 =1.0, c=1e-4,    =0.5, max_ls=50)
       =  0
    gg = dot(g, g)
    it = 0
    while it < max_ls
        xn = x .-     .* g
        fn = f(xn)
        if fn <= fx - c *    * gg
            return xn, fn,    , it
        end
           *=
        it += 1
    end
    xn = x .-     .* g
    return xn, f(xn),    , it
end

function gradient_descent(f, gradf, x0; tol_g=1e-10, tol_x=1e-12, max_iter=50_000,  0 =1.0)
    x = copy(x0)
    fx = f(x)
    it = 0
       = 0.5
    while it < max_iter
        g = gradf(x)
        ng = sqrt(dot(g, g))
        if ng <= tol_g
            break
        end
        xn, fn,    , _ = backtracking_step(f, x, fx, g;  0 = 0 ,    = )
        if sqrt(dot(xn .- x, xn .- x)) <= tol_x
            x = xn
            fx = fn
```

```
            break
        end
        x = xn
        fx = fn
         0  = min(1.0,     /    )
        it += 1
    end
    return x, fx, it
end

function exterior_penalty_method_2d(f, gradf, g_list::Vector{Function}, gradg_list::Vector{Function}, x0;
    r0=1.0, beta=10.0, tol_x=1e-10, tol_violation=1e-12, max_outer=20, inner_tol_g=1e-10, inner_tol_x=1e-12, max_inner=50_000)
    r = fill(r0, length(g_list))
    x_prev = copy(x0)
    outer = 0
    while outer < max_outer
        function   (x)
            s = f(x)
            for j in eachindex(g_list)
                gj = g_list[j](x)
                s += r[j] * ((gj > 0) ? (gj * gj) : 0.0)
            end
            return s
        end
        function grad (x)
            g = gradf(x)
            for j in eachindex(g_list)
                gj = g_list[j](x)
                if gj > 0
                    g .+= (2r[j] * gj) .* gradg_list[j](x)
                end
            end
            return g
        end
        x_new, _, _ = gradient_descent(  , grad  , x_prev; tol_g=inner_tol_g, tol_x=inner_tol_x, max_iter=max_inner)
        v = 0.0
        for j in eachindex(g_list)
            gj = g_list[j](x_new)
            v += (gj > 0) ? (gj * gj) : 0.0
        end
        if sqrt(dot(x_new .- x_prev, x_new .- x_prev)) <= tol_x && v <= tol_violation
```

```
            x_prev = x_new
            break
        end
        x_prev = x_new
        r .*= beta
        outer += 1
    end
    return x_prev, f(x_prev), outer, r
end

function try_using_plots()
    try
        @eval using Plots
        return true
    catch
        return false
    end
end

function ellipse_shape(cx, cy, rx, ry; n=220)
    xs = Vector{Float64}(undef, n)
    ys = Vector{Float64}(undef, n)
    for i in 1:n
            = 2pi * (i - 1) / (n - 1)
        xs[i] = cx + rx * cos(  )
        ys[i] = cy + ry * sin(  )
    end
    return xs, ys
end

function visualize_1d_example(; r_list=[1.0, 10.0, 100.0, 10_000.0, 100_000.0, 1_000_000.0], x_range=(-0.5, 3.5), saveprefix="lab13_1d")
    ok = try_using_plots()
    f(x) = x * x
    g(x) = 2 - x
    Qr(x, r) = f(x) + r * ((g(x) > 0) ? (g(x) * g(x)) : 0.0)
    xs = range(x_range[1], x_range[2]; length=1000)
    mins = Tuple{Float64,Float64,Float64}[]
    for r in r_list
            = x -> Qr(x, r)
        x_star, y_star, _ = golden_section_search(  , x_range[1], x_range[2]; tol=1e-12)
        push!(mins, (r, x_star, y_star))
    end
```

```
    if ok
        for (idx, r) in enumerate(r_list)
            ys = [Qr(x, r) for x in xs]
            plt = Plots.plot(; title="Exterior penalty (1D), r=$(r)", xlabel="x", ylabel="Q(x,r)", legend=:topright)
            Plots.vspan!(plt, [2.0, x_range[2]]; label="feasible x>=2", color=:green, alpha=0.08)
            Plots.plot!(plt, xs, ys; label="Q(x,r)")
            _, x_star, y_star = mins[idx]
            Plots.scatter!(plt, [x_star], [y_star]; label="x*")
            Plots.vline!(plt, [2.0]; label="x=2")
            Plots.savefig(plt, "$(saveprefix)_r$(r).png")
        end
    end
    return mins, ok
end

function visualize_2d_example(; alpha=1.0, beta=2.0, use_g2=true, r_list=[1.0, 10.0, 100.0, 10_000.0, 100_000.0, 1_000_000.0], x0=[1.2, 1.0], saveprefix="lab13_2d")
    ok = try_using_plots()
    f(x) = alpha * x[1]^2 + beta * x[2]^2
    gradf(x) = [2alpha * x[1], 2beta * x[2]]
    g_list = Function[]
    gradg_list = Function[]
    push!(g_list, x -> 2 - x[1])
    push!(gradg_list, x -> [-1.0, 0.0])
    if use_g2
        push!(g_list, x -> 2 - x[2])
        push!(gradg_list, x -> [0.0, -1.0])
    end
    function is_feasible(x)
        for gj in g_list
            if gj(x) > 0
                return false
            end
        end
        return true
    end
    x0_use = copy(x0)
    if is_feasible(x0_use)
        x0_use = use_g2 ? [1.2, 1.0] : [1.2, 2.4]
    end

    xs = Vector{Vector{Float64}}()
```

```
    x = copy(x0_use)
    for r in r_list
        x, _, _, _ = exterior_penalty_method_2d(f, gradf, g_list, gradg_list, x; r0=r, beta=10.0, max_outer=1, tol_x=1e-14, tol_violation=0.0, inner_tol_g=1e-12, inner_tol_x=1e-14, max_inner=200_000)
        push!(xs, copy(x))
    end
    if ok
        function Qr2(x, r)
            s = f(x)
            for gj in g_list
                v = gj(x)
                s += r * ((v > 0) ? (v * v) : 0.0)
            end
            return s
        end
        x1s = range(0.0, 3.5; length=250)
        x2s = range(0.0, 3.5; length=250)
        ex2, ey2 = ellipse_shape(2.0, 2.0, 0.45, 0.45)
        oval2 = Plots.Shape(ex2, ey2)

        pathx = [x0_use[1]; [p[1] for p in xs]]
        pathy = [x0_use[2]; [p[2] for p in xs]]

        plts = Any[]
        for r in r_list
            Z = [Qr2([x1, x2], r) for x2 in x2s, x1 in x1s]
            plt = Plots.contour(x1s, x2s, Z; levels=30, fill=true, c=:viridis, xlabel="x1", ylabel="x2", title="r=$(r)")
            Plots.plot!(plt, oval2; label=false, color=:green, alpha=0.12, linecolor=:green)
            Plots.vline!(plt, [2.0]; label=false, lc=:white, ls=:dash)
            if use_g2
                Plots.hline!(plt, [2.0]; label=false, lc=:white, ls=:dash)
            end
            Plots.plot!(plt, pathx, pathy; label=false, lc=:red, lw=2)
            Plots.scatter!(plt, pathx, pathy; label=false, mc=:red, ms=4)
            push!(plts, plt)
            Plots.savefig(plt, "$(saveprefix)_contour_r$(r).png")
        end
```

```
        plts = nothing

        Zf = [f([x1, x2]) for x2 in x2s, x1 in x1s]
        for (i, r) in enumerate(r_list)
            tr = Plots.contour(x1s, x2s, Zf; levels=30, fill=false, c=:grays, xlabel="x1", ylabel="x2", title="Trajectory for r=$(r)")
            Plots.plot!(tr, oval2; label=false, color=:green, alpha=0.12, linecolor=:green)
            Plots.vline!(tr, [2.0]; label=false, lc=:black, ls=:dash)
            if use_g2
                Plots.hline!(tr, [2.0]; label=false, lc=:black, ls=:dash)
            end
            px = [x0_use[1], xs[i][1]]
            py = [x0_use[2], xs[i][2]]
            Plots.plot!(tr, px, py; label=false, lc=:red, lw=2)
            Plots.scatter!(tr, px, py; label=false, mc=:red, ms=4)
            Plots.savefig(tr, "$(saveprefix)_trajectory_r$(r).png")
        end
        dirs = Dict(
            "x1" => [1.0, 0.0],
            "x2" => [0.0, 1.0],
            "diag" => [1/sqrt(2), 1/sqrt(2)]
        )
        t = range(-2.5, 2.5; length=1200)
        for (name, d) in dirs
            plt = Plots.plot(; title="Slices from x0 along $(name)", xlabel="t", ylabel="Q(x0+t*d, r)", legend=:topright)
            for r in r_list
                ys = [begin
                    x = x0_use .+ tt .* d
                    Qr2(x, r)
                end for tt in t]
                Plots.plot!(plt, t, ys; label="r=$(r)")
            end
            Plots.savefig(plt, "$(saveprefix)_slice_$(name).png")
        end
    end
    return xs, ok
end

function main()
```

```
    mins, ok1 = visualize_1d_example(; r_list=[1.0, 10.0, 100.0, 10_000.0, 100_000.0, 1_000_000.0], x_range=(-0.5, 3.5), saveprefix="lab13_1d")
    for (r, x, q) in mins
        println("1D: r=", r, "  x*=", x, "  Q=", q)
    end
    if ok1
        println("Saved: lab13_1d_r*.png")
    else
        println("Plots not available, 1D plot skipped")
    end

    xs, ok2 = visualize_2d_example(; alpha=1.0, beta=2.0, use_g2=true, r_list=[1.0, 10.0, 100.0, 10_000.0, 100_000.0, 1_000_000.0], x0=[1.2, 1.0], saveprefix="lab13_2d")
    for (i, x) in enumerate(xs)
        println("2D: step=", i, "  x=[", x[1], ", ", x[2], "]")
    end
    if ok2
        println("Saved: lab13_2d_contour_r*.png, lab13_2d_trajectory_r*.png, lab13_2d_slice_*.png")
    else
        println("Plots not available, 2D plots skipped")
    end
end

main()
```

Результат представлен на рисунках 1– 2.

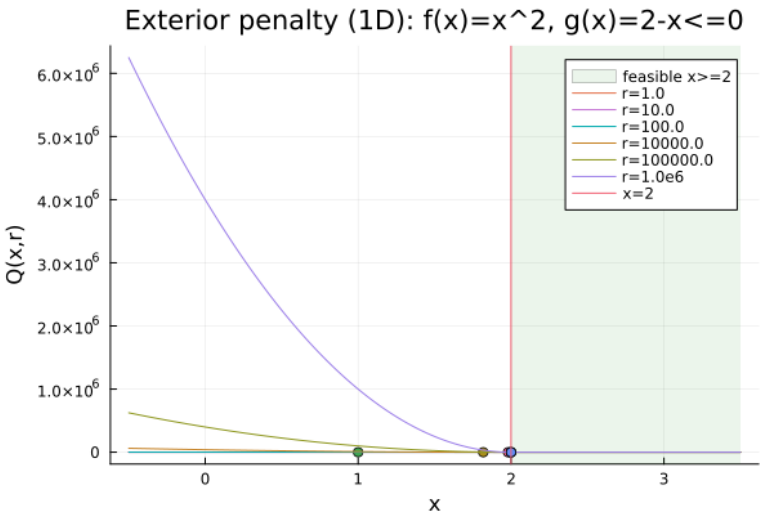

Рис. 1 — Результат

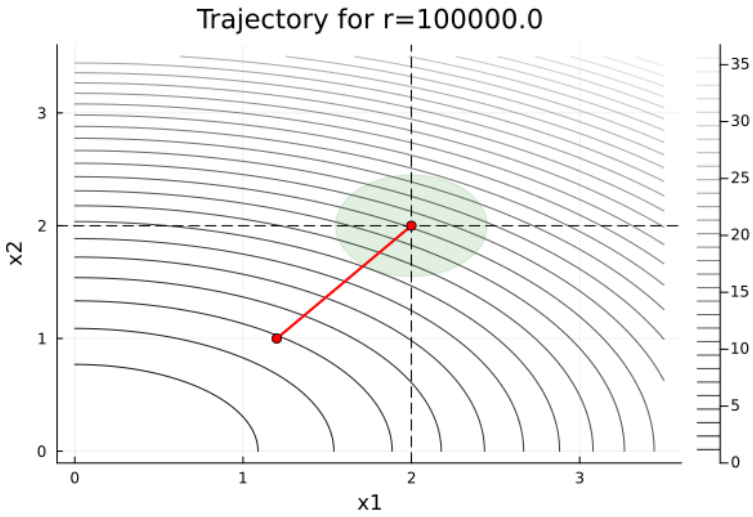

Рис. 2 — Результат